Important DSA Viva Questions & Answers

Array & Strings

Q: Difference between array and linked list?

A: Arrays are fixed-size and allow random access; linked lists are dynamic-size and allow efficient insertions/deletions.

Q: How do you reverse an array in-place?

A: Swap elements from both ends moving toward the center using two pointers.

Q: Time complexity for access/insert/delete in array?

A: Access: O(1), Insert/Delete: O(n) (due to shifting).

Q: What is the sliding window technique?

A: A technique to maintain a subset of elements over a moving window to reduce complexity from O(n) to O(n).

Q: Difference between char array and string in C++?

A: Char array is a C-style string; `string` is a C++ class with built-in functions.

Linked List

Q: Types of linked lists?

A: Singly, Doubly, and Circular linked lists.

Q: How to detect a cycle in a linked list?

A: Use Floyd's cycle detection (slow and fast pointers).

Q: How to reverse a linked list?

A: Iteratively: track `prev`, `curr`, and `next` pointers.

Q: Advantages of linked list over array?

A: Dynamic sizing and efficient insertions/deletions.

Q: Find middle of linked list?

A: Use slow and fast pointers. When fast reaches end, slow is at the middle.

Stacks and Queues

Q: What is a stack?

A: A LIFO (Last In First Out) data structure.

Q: Difference between stack and queue?

A: Stack is LIFO; queue is FIFO (First In First Out).

Q: Queue using two stacks?

A: Push to one stack; for pop, reverse elements into second stack.

Q: Time complexity of push/pop in stack?

A: O(1) for both.

Q: What is a circular queue?

A: A queue that wraps around to use unused space at the beginning.

Trees

Q: Binary tree vs BST?

A: In BST, left < root < right; binary tree has no such condition.

Q: Inorder/Preorder/Postorder?

A: Inorder: LNR, Preorder: NLR, Postorder: LRN.

Q: Check if a tree is balanced?

A: Recursively check if left and right subtrees differ in height by no more than 1.

Q: Height of a tree?

A: Number of edges on the longest path from root to leaf.

Q: DFS vs BFS?

A: DFS uses stack/recursion; BFS uses queue. DFS goes deep, BFS explores level by level.

Hashing

Q: What is hashing?

A: A technique to map keys to values using a hash function.

Q: Hash collision and resolution?

A: When two keys hash to the same index. Resolved via chaining or open addressing.

Q: Applications of hash tables?

A: Caching, dictionaries, sets, etc.

Q: Why is `unordered_map` faster than `map` in C++?

A: Uses hashing (average O(1)); map uses BST (O(log n)).

Q: Time complexity in hash table?

A: Average: O(1), Worst-case: O(n) due to collisions.

Dynamic Programming

Q: What is DP?

A: Solving complex problems by breaking them into overlapping subproblems and storing results.

Q: Memoization vs Tabulation?

A: Memoization: Top-down + recursion + cache. Tabulation: Bottom-up + iteration.

Q: Example problems?

A: Fibonacci, 0/1 Knapsack, Longest Common Subsequence.

Q: Identify DP problem?

A: Look for optimal substructure and overlapping subproblems.

Q: Fibonacci with DP complexity?

A: Time: O(n), Space: O(n) or O(1) with optimization.

Recursion & Backtracking

Q: What is recursion?

A: A function calling itself.

Q: Base condition?

A: A stopping condition to end recursive calls.

Q: Recursion vs Iteration?

A: Recursion uses function stack; iteration uses loops.

Q: What is backtracking?

A: A technique to explore all solutions by building a solution incrementally and removing it if it doesnt work.

Q: Example?

A: N-Queens, Sudoku solver, Subset generation.

Complexity Analysis

Q: What is time/space complexity?

Important DSA Viva Questions & Answers

A: Time: How fast code runs. Space: Memory used.

Q: Meaning of O(1), O(n), O(n), O(log n)?

A: Constant, linear, quadratic, logarithmic time complexities.

Q: Why is O(n log n) efficient for sorting?

A: It strikes a balance between speed and reliability (merge/quick sort).

Q: Nested loop complexity?

A: Multiply inner and outer loop sizes (e.g., O(n) * O(n) = O(n)).

Q: Best, worst, average case?

A: Best: Fastest execution; Worst: Slowest; Average: Expected performance.